

The methodology of **SatQuery AI** is structured around a **typed, schema-validated agentic execution graph** operating above a shared, parameter-efficient **Vision-Language Model (VLM)** backbone 1-3. Rather than relying on an unconstrained Large Language Model (LLM) or a fragile free-form ReAct loop, SatQuery AI enforces a **"validate before you execute"** architectural pattern 1, 2, 4.

1. Model Architecture & Remote-Sensing Adaptation

SatQuery AI avoids multi-task collapse and high memory overhead by building a **modular LoRA adapter bank** over a single, frozen 1–4B open-weight VLM backbone 3, 5-7:

- **Shared Frozen Backbone:** Uses a compact 1–4B VLM (such as an InternVL3-1B-class or small Qwen-VL variant) 6. The vision encoder (e.g., CLIP-ViT) and base LLM decoder remain completely frozen 6, 8.
- **Modality Projection Bridges:** Small linear projection layers map Sentinel-1 Synthetic Aperture Radar (SAR) and Sentinel-2 multispectral tokens directly into the language decoder's embedding space 6, 8.
- **Task-Specific LoRA Adapters (Rank 8–16):** Fine-tuned on less than 1% of total model parameters using Low-Rank Adaptation 6, 8:
- **adapter_vqa_caption (Joint VQA & Captioning):** Fine-tuned on BigEarthNet.txt for open-ended/closed-set Visual Question Answering and scene descriptions 6, 8.
- **adapter_grounding (Dedicated Region Grounding):** Fine-tuned separately on referring expressions to output bounding-box token sequences 6, 8, 9. Isolating grounding into its own adapter prevents its bounding-box token formatting requirements from interfering with natural-language text generation 6, 8.
- **adapter_change_vqa (Bi-Temporal Change Analysis):** Fine-tuned on the CDVQA dataset alongside a difference-attention **Change-Enhancing Module (CEM)** that compares Siamese-encoded T1 and T2 feature representations 10, 11.
- **Data-Quality Pre-Filtering:** Training pairs from BigEarthNet.txt pass through a **ScoreRS-style quality-scoring filter** prior to fine-tuning, dropping low-quality pairs to improve downstream performance 6, 12, 13.

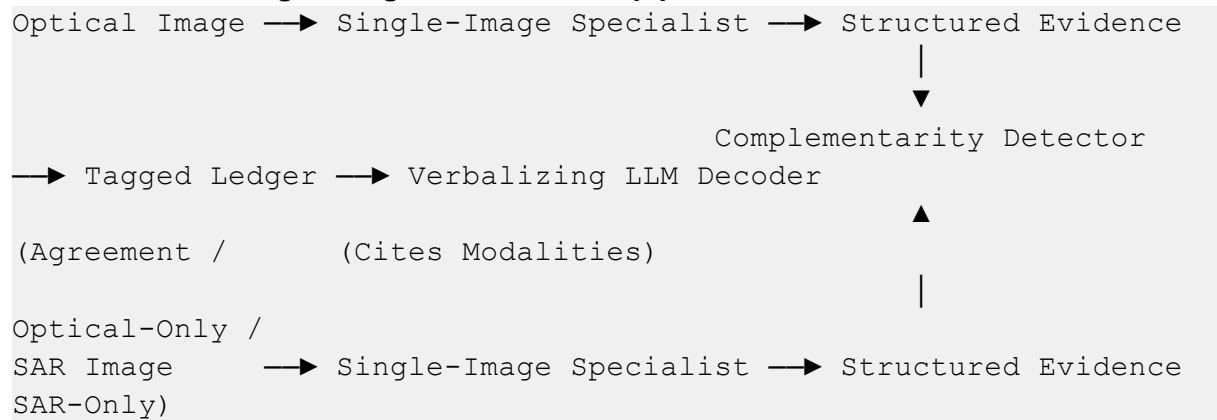
2. Domain-Gap Mitigation Pipeline

A central engineering requirement of SatQuery AI is mitigating the resolution and sensor gap between the mandatory training dataset (**Sentinel-1/2**, 10–60 m resolution) and high-resolution evaluation datasets (**Cartosat-2S optical**, 0.65–2 m resolution, and **RISAT SAR**) 1, 4, 14, 15:

1. **GSD-Normalization Tiling:** Rescales spatial dimensions and applies tiled sampling to align disparate Ground Sample Distances 4, 15, 16.
2. **Radiometric-Jitter Augmentation:** Applies histogram matching toward Cartosat-2S response curves and injects calibrated speckle noise matching RISAT SAR backscatter characteristics 15, 16.
3. **Multi-Resolution Alignment:** Incorporates multi-resolution datasets (such as **SOMA-1M**) during optical-SAR feature fusion to bridge resolution regimes 12, 16, 17.

3. Optical-SAR Cross-Modal Fusion Pipeline

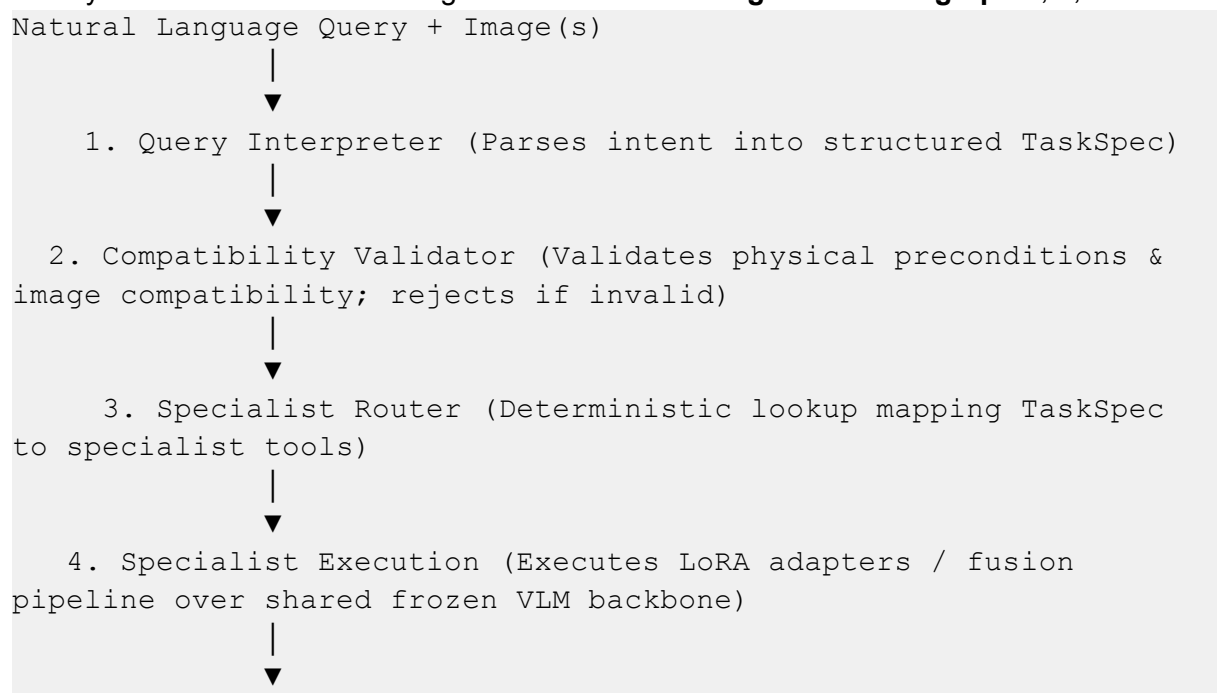
To overcome the documented failure modes of naive early-fusion channel concatenation (which regresses performance relative to RGB-only baselines), SatQuery AI employs a **structured, two-stage non-generative fusion pipeline** 4, 15, 18-20:

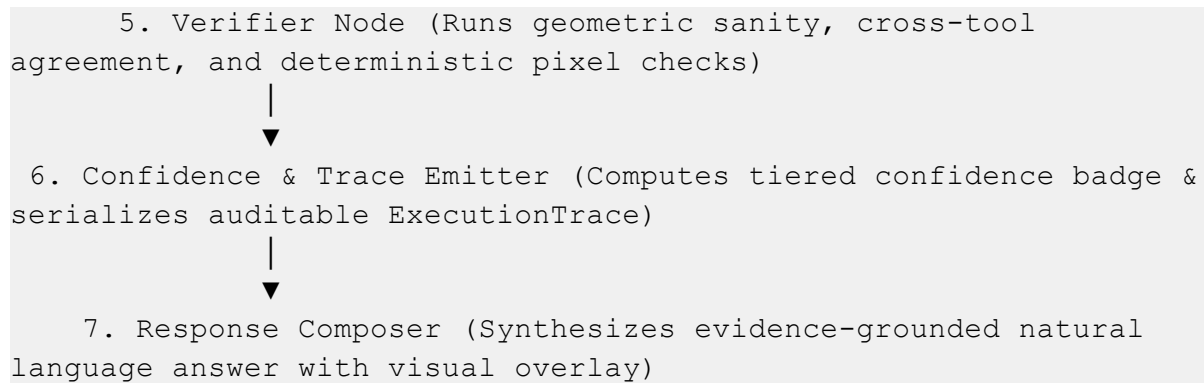


1. **Independent Evidence Extraction:** The single-image specialist extracts structured land-use/land-cover (LULC) evidence (class presence, area shares, grounded regions) independently from the optical and SAR images 15, 20.
2. **Complementarity Detection:** A lightweight classifier trained on BigEarthNet.txt S1/S2 pairs using **InfoNCE contrastive alignment** (MM-OVSeg recipe) tags regions as agreement, optical_only, or sar_only 15, 21.
3. **Auditable Verbalization:** The controller LLM synthesizes the tagged evidence into a natural-language description, explicitly citing which modality provided each specific claim (e.g., identifying water bodies obscured by clouds in optical imagery via SAR backscatter) 15, 22.

4. Agentic Model and Tool Orchestration

The system executes tasks through a deterministic **7-stage execution graph** 2, 3, 23:





Step 1: Query Interpreter

A text-only instruct LLM parses user queries and image metadata into a structured TaskSpec JSON object using constrained function calling 2, 24. If intent confidence falls below 0.55, it triggers a clarifying question rather than guessing 24.

Step 2: Compatibility Validator

A non-LLM, deterministic node that checks TaskSpec requirements against machine-readable physical preconditions before any specialist model runs 2, 25, 26:

- **Format & Modality Validation:** Validates file formats (GeoTIFF, TIFF, PNG, JPEG) and sensor types 26, 27.
- **Geospatial & Temporal Preconditions:** Checks Spatial Reference Systems (CRS), calculates spatial footprint overlap via Intersection over Union (IoU $\geq 70\%$), and verifies acquisition date ordering for change tasks 26-28.

If a query is physically unsatisfiable (e.g., requesting bi-temporal change detection on non-overlapping footprints), the validator halts execution upfront with a **ValidationRejection** reason code and human-readable explanation 26, 28, 29.

Step 3: Specialist Router

Maps the validated TaskSpec to specialized tools via a fixed, deterministic lookup table (e.g., mapping fusion_then_change to a sequential execution of the fusion pipeline followed by the change-VQA specialist) 2, 30.

Step 4: Specialist Execution

Executes the assigned task using **adapter hot-swapping** on model_serving 7, 31. All LoRA deltas (<50 MB each) remain resident in GPU memory simultaneously, allowing active deltas to be swapped per forward pass without reloading the base backbone 7, 31.

Step 5: Verifier Node

Performs cross-checks on collected evidence prior to user presentation 2, 32:

- **Geometric Sanity:** Verifies bounding box coordinates lie strictly within normalized $[0, 1]$ bounds and change masks do not exceed overlapping footprints 2, 32.
- **Deterministic Quantity Cross-Check:** For counting or ratio queries, extracts deterministic pixel/instance counts from change masks and compares them against verbalized LLM claims 5, 10, 32, 34. Discrepancies beyond a $\pm 15\%$ tolerance band force an automatic confidence downgrade 32, 35.

Step 6: Confidence & Trace Emitter

Computes a **High**, **Medium**, or **Low** confidence badge using **decoupled perception and reasoning confidence tokens** 2, 35-37:

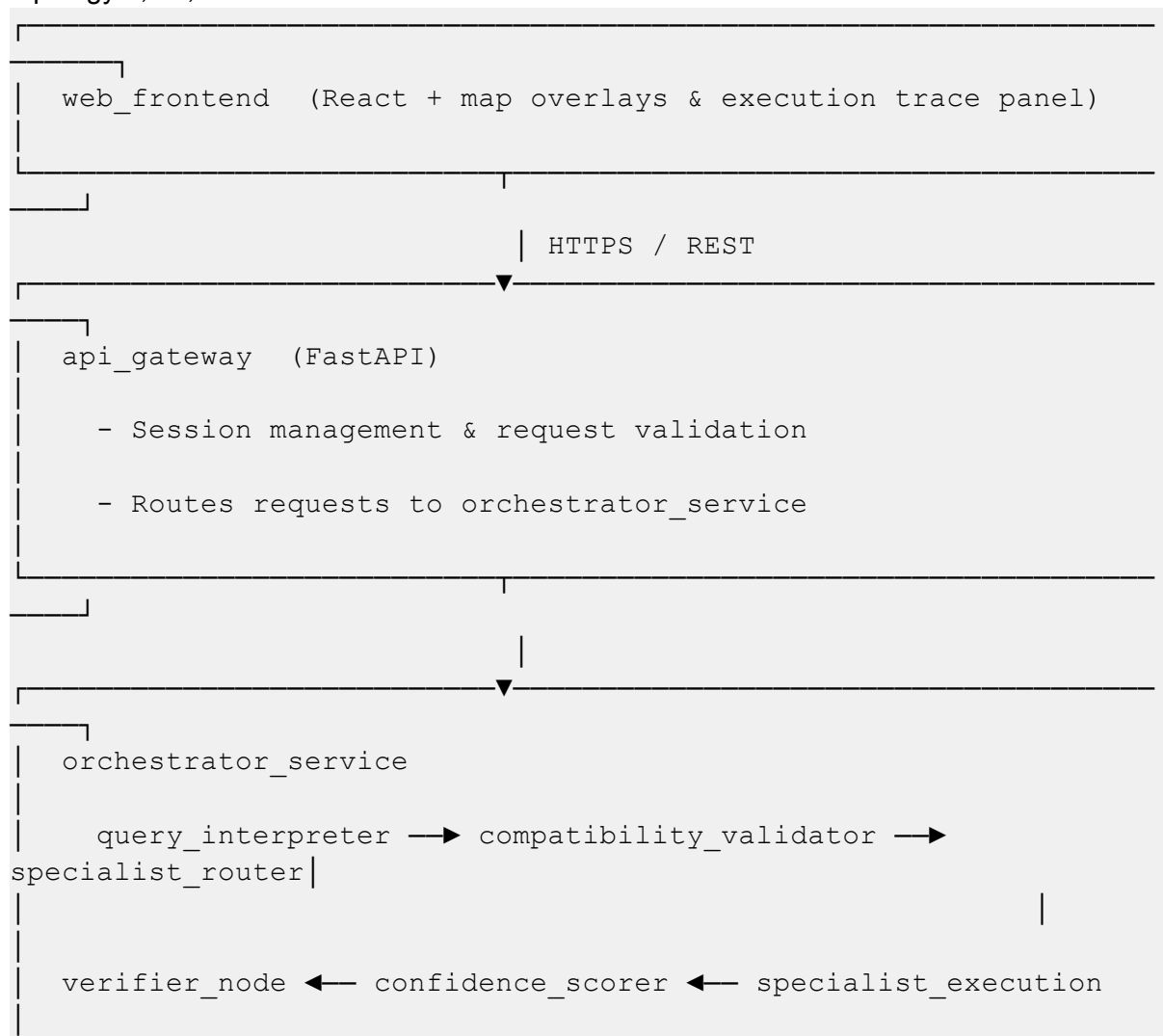
- Applies structural discounts for documented weak points (e.g., counting or small-change queries) 10, 35, 38.
- Applies hard overrides to **Low** if the Verifier Node detects geometric violations or quantity discrepancies 2, 35.
- Serializes a complete, auditable **ExecutionTrace** logging tasks, tools, parameters, timing, and outputs 2, 33, 39.

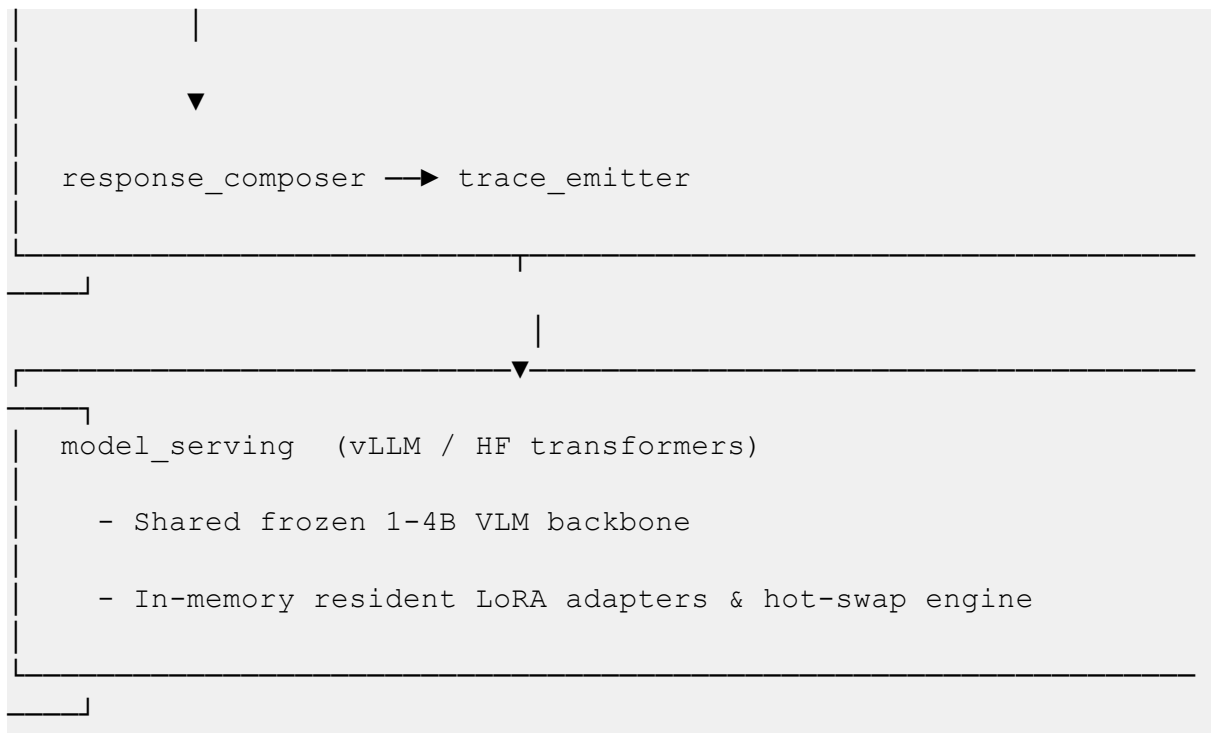
Step 7: Response Composer

Synthesizes the verified evidence into a final natural-language response accompanied by visual overlays (bounding boxes, change masks) and downloadable PDF/JSON execution reports 2, 23, 40, 41.

5. System Architecture & Serving Topology

SatQuery AI is organized into modular services running in an offline-capable container topology 3, 42, 43:





- **web_frontend:** React interface featuring map-based evidence overlays, capability hints, collapsible execution trace panels, and downloadable reports 3, 43, 44.
- **api_gateway:** FastAPI entry point handling session tokens and multipart upload routing 3, 43, 45.
- **orchestrator_service:** Manages graph execution from interpretation and validation through verification and trace serialization 3, 43.
- **model_serving:** GPU-attached process hosting the frozen backbone and resident LoRA deltas, executing forward passes with zero-reload adapter swapping 3, 31, 43.
- **object_store & metadata_db:** Stores raw rasters, normalized tiles, derived overlays, session records, and serialized evidence ledgers 3, 46, 47.

💡 *Would you like to review the specific unit testing strategy for the Compatibility Validator, or examine how the frontend renders multi-candidate grounding bounding boxes?*